Rômulo Maurício Fonseca Júnior - 15553801
EP2 - Computação Orientada a Objetos

O projeto foi submetido a um processo de refatoração e modernização com o objetivo de melhorar sua estrutura, legibilidade e extensibilidade, resolvendo os problemas identificados na versão original. As principais mudanças foram o uso de Coleções Java e a aplicação dos padrões de projeto

Strategy e *Decorator*.

1. Modernização do Código: Adoção do Arcabouço de Coleções Java

- **Mudança:** A estrutura de dados principal do sistema foi alterada de vetores (`Produto[]`) para o uso de `List<Produto>`, uma das interfaces do arcabouço de coleções do Java.
- **Justificativa:** A versão original não seguia as convenções modernas da linguagem Java. O uso de `List` oferece maior flexibilidade (tamanho dinâmico), um conjunto rico de métodos de manipulação (como os da classe `Collections`) e melhora a integração com outras partes da linguagem. Isso resolveu a rigidez dos vetores e tornou o código mais limpo e convencional.

2. Aplicação do Padrão de Projeto *Strategy*

O padrão

Strategy foi utilizado para encapsular os comportamentos que variavam na geração do relatório: os algoritmos de ordenação, os critérios de ordenação e os critérios de filtragem.

- **Problema Resolvido:** No código original, toda a lógica para os diferentes algoritmos e critérios estava centralizada na classe `GeradorDeRelatorios`, provavelmente em blocos `if/else` ou `switch`. Isso tornava a adição de novas opções uma tarefa complexa e propensa a erros, exigindo modificação em código já existente.
- **Solução Implementada:**
    - **Filtragem:** Foi criada a interface `FiltroStrategy` com um método `verificar(Produto p)`. Classes concretas como `FiltroTodos`, `FiltroCategoriaIgualA` e as novas `FiltroPrecoDentroDeFaixa` e `FiltroDescricaoContemSubstring` implementam essa interface, cada uma contendo a lógica de um critério específico.
    - **Ordenação:** A ordenação foi dividida em duas partes:
        1. **Critérios:** Foram implementados como classes que utilizam a interface nativa `java.util.Comparator<Produto>`, como `CriterioPrecoCrescente` e `CriterioDescricaoDecrescente`.

2. **Algoritmos:** Foi criada a interface `AlgoritmoOrdenacao` e as classes `QuickSort` e `InsertionSort` a implementam. Elas foram adaptadas para operar sobre uma `List` e usar um `Comparator` para realizar as comparações, desacoplando o algoritmo do critério.

- **Vantagens:** A classe `GeradorDeRelatorios` deixou de conhecer os detalhes de cada algoritmo ou critério. Ela agora apenas recebe e utiliza os objetos de estratégia, tornando o sistema extensível e alinhado ao Princípio Aberto/Fechado. Adicionar novas funcionalidades de filtro ou ordenação agora se resume a criar novas classes de estratégia, sem alterar o gerador.

3. Aplicação do Padrão de Projeto *Decorator*

O padrão

Decorator foi aplicado para gerenciar as opções de formatação do texto (negrito, itálico, etc.).

- **Problema Resolvido:** Na versão original, a formatação era responsabilidade do `GeradorDeRelatorios` e era aplicada de forma uniforme a todos os produtos da lista resultante. Não era possível combinar múltiplas formatações de forma flexível ou aplicar formatações diferentes a produtos específicos.
- **Solução Implementada:**
  - Foi criado um `ProdutoDecorator` abstrato que implementa a interface `Produto`. Isso permite que um "produto decorado" seja tratado da mesma forma que um produto normal.
  - Foram criadas classes de decorador concretas (`ProdutoEmNegrito`, `ProdutoEmItalico`, `ProdutoCor`) que "envelopam" um objeto `Produto`.
  - Cada decorador modifica a saída do método `formataParaImpressao`, adicionando sua própria tag HTML (`<b>`, `<i>`, `<font>`) ao redor do resultado do produto que ele envelopa.
- **Vantagens:** A responsabilidade de formatação foi movida para o domínio do `Produto`. Agora é possível aplicar múltiplas formatações a um mesmo produto dinamicamente (ex: negrito e itálico ao mesmo tempo) e destacar produtos individualmente, atendendo a um requisito chave da refatoração.

4. Implementação de Novas Funcionalidades

Como parte da melhoria do sistema, as seguintes funcionalidades foram adicionadas, aproveitando a nova arquitetura flexível:

- **Ordenação:** Três novos critérios em ordem decrescente (descrição, preço, estoque).
- **Filtragem:** Dois novos critérios (preço por intervalo e descrição contendo substring).
- **Formatação:** Um novo decorador para definir a cor do texto.
- **Carregamento de Dados:** Uma classe utilitária foi criada para carregar os produtos de um arquivo CSV, salvando-os em uma coleção.